

- Produkcja P12 – dokumentacja techniczna (PolyDPG)
 - 1. Cel produkcji
 - 2. Kontekst: model grafu
 - 2.1 Klasa Node
 - 2.2 Klasa HyperEdge
 - 2.3 Klasa Graph
 - 3. Produkcja P12
 - 4. Metody produkcji
 - 4.1 `can_apply(self, graph)`
 - 4.2 `find_match(self, graph)`
 - 4.3 `get_left_side(self)`
 - 4.4 `get_right_side(self, matched, level)`
 - 5. Funkcje pomocnicze do testów
 - 6. Testy jednostkowe (unittest)
 - 6.1 Testy izomorfizmu
 - 6.2 Testy poprawności wykonania produkcji

Produkcja P12 – dokumentacja techniczna (PolyDPG)

1. Cel produkcji

Produkcja P12 służy do oznaczania siedmiokątnego elementu grafu (hiperkrawędź typu **T** z dokładnie 7 wierzchołkami) do dalszego *refinementu* poprzez ustawienie atrybutu:

$R = 1$

Produkcja:

- działa **lokalnie** (na podgrafie),
- **nie zmienia struktury grafu**,
- **nie zmienia osadzenia geometrycznego**,
- **nie usuwa ani nie dodaje wierzchołków**,
- modyfikuje wyłącznie atrybut **R** hiperkrawędzi **T**.

2. Kontekst: model grafu

2.1 Klasa **Node**

Reprezentuje wierzchołek grafu.

Atrybut	Opis
<code>x, y</code>	współrzędne geometryczne
<code>label</code>	etykieta wierzchołka
<code>hanging</code>	czy węzeł jest wiszący
<code>hyperref</code>	referencja do hiperkrawędzi (jeśli węzeł ją reprezentuje)

Równość i haszowanie oparte są **wyłącznie na etykiecie**.

2.2 Klasa **HyperEdge**

Reprezentuje hiperkrawędź jako węzeł pomocniczy.

Atrybut	Opis
<code>nodes</code>	krotka wierzchołków
<code>hypertag</code>	typ (<code>T</code> , <code>E</code> , ...)
<code>R</code>	znacznik refinementu
<code>B</code>	znacznik brzegowy

Hiperkrawędź:

- musi zawierać co najmniej 2 wierzchołki,
- jest porównywana strukturalnie (tag + zbiór węzłów).

2.3 Klasa **Graph**

Kontener na:

- zbiór wierzchołków (**Node**)

- zbiór hiperkrawędzi (**HyperEdge**)
- wewnętrzny graf **networkx**

Najważniejsze metody:

- **add_node**
- **add_edge**
- **remove_node**
- **remove_edge**
- **apply(production)**

Metoda **apply**:

1. sprawdza **can_apply**
2. wycina dopasowany podgraf
3. usuwa go z grafu głównego
4. wstawia graf prawej strony produkcji

3. Produkcja **P12**

```
@Production.register  
class P12(Production):
```

Produkcja jest automatycznie rejestrowana w globalnym rejestrze produkcji.

4. Metody produkcji

4.1 **can_apply(self, graph)**

Sprawdza, czy w grafie istnieje podgraf izomorficzny z lewą stroną produkcji.

Zwraca:

- **True** – jeśli znaleziono poprawny siedmiokąt
- **False** – w przeciwnym przypadku

4.2 find_match(self, graph)

Najważniejsza metoda produkcji – pełne sprawdzenie izomorfizmu. Warunki dopasowania:

- Istnieje hiperkrawędź:
- `hypertag == "T"`
- `R == 0`
- dokładnie 7 wierzchołków
- Wszystkie wierzchołki należą do grafu
- Istnieje dokładnie 7 hiperkrawędzi typu E
- Każda E:
 - łączy dokładnie 2 wierzchołki
 - należy do zbioru wierzchołków T
- Graf E jest jednym cyklem (spójność)

Jeśli wszystkie warunki są spełnione → graf jest izomorficzny z lewą stroną produkcji.

4.3 get_left_side(self)

Buduje abstrakcyjny graf lewej strony produkcji. Struktura:

- 7 wierzchołków `v0–v6`
- 1 hiperkrawędź: `T(v0...v6)`, `R = 0`
- 7 hiperkrawędzi E tworzących cykl

Graf ten:

- jest wzorcem izomorfizmu,
- wykorzystywany jest w testach.

4.4 get_right_side(self, matched, level)

Tworzy graf prawej strony produkcji.

Działanie:

- kopiuje wszystkie wierzchołki
- kopiuje wszystkie hiperkrawędzie
- zachowuje etykiety,
- zachowuje współrzędne,
- nie zmienia osadzenia grafu.

5. Funkcje pomocnicze do testów

make_septagon() Tworzy graf izomorficzny z lewą stroną produkcji.

make_big_graph_with_septagon_subgraph() Tworzy większy graf zawierający:

poprawny siedmiokąt jako podgraf

dodatkowe wierzchołki niezwiązane z produkcją

Służy do testowania lokalności produkcji.

6. Testy jednostkowe (unittest)

Klasa testowa:

python Skopiuj kod class TestP12Isomorphism(unittest.TestCase):

6.1 Testy izomorfizmu

Test	Sprawdzana własność
test_can_apply_to_isomorphic_graph	poprawny graf

Test	Sprawdzana własność
test_missing_vertex_breaks_isomorphism	brak wierzchołka
test_missing_edge_breaks_isomorphism	brak krawędzi
test_relabeling_vertex_breaks_isomorphism	etykiety nie wpływają
test_septagon_as_subgraph_of_larger_graph	poprawność dla podgrafu

6.2 Testy poprawności wykonania produkcji

Test	Sprawdzana własność
test_apply_does_not_damage_supergraph	brak uszkodzeń grafu nadrzędnego
test_embedding_transformation	R zmienia się z 0 $\rightarrow$ 1
test_rhs_graph_structure	poprawna struktura RHS
test_vertex_coordinates_preserved	zachowanie współrzędnych